



Departamento de Computación e Informática
Computación Paralela y Distribuida

Trabajo de análisis computación paralela

Informe Técnico — Trabajo Práctico

Integrantes: Matías Fernández López

Camilo Moya

Ignacio Ortega Parra

Profesor: Sebastián Salazar

Entrega: 10 de julio de 2026

Índice

1. Introducción	4
2. Descripción de la Solución Implementada	5
2.1 Justificación de las librerías utilizadas	5
3. Datos y Preprocesamiento	6
3.1 Carga y manejo de memoria	6
3.2 Tratamiento de valores faltantes	6
3.3 Detección de outliers	6
3.4 Transformación y creación de variables derivadas	7
3.5 Estandarización de variables	7
4. Análisis Exploratorio Estadístico	9
4.1 Estadística descriptiva	9
4.2 Tests de normalidad	9
4.3 Visualizaciones	10
Histogramas con curva de densidad	10
Boxplot por categoría	12
Matrices de correlación	12
4.4 Análisis de asociación	13
Variables numéricas: correlación con prueba de significancia	13
Variables categóricas: independencia Chi-cuadrado	13
Análisis de varianza (ANOVA)	14
4.5 Patrones temporales	14
Descomposición temporal (tendencia, estacionalidad, residuo)	15
Autocorrelación (ACF) y autocorrelación parcial (PACF)	15
5. Inferencia Estadística y Modelado	16
5.1 Pruebas de hipótesis	16
H1 (obligatoria). El ticket promedio en APP es mayor que en WEB	16
H2 (obligatoria). El % de descuento afecta significativamente las unidades vendidas	16
H3 (propia). El monto de compra difiere significativamente según el género del cliente	16
H4 (propia). Existe asociación entre el canal de compra y el local	17
H5 (propia). La edad del cliente afecta significativamente el monto de compra	17
5.2 Modelado predictivo — Opción A: Regresión	18
Diagnóstico de supuestos	18
Coeficientes del modelo	18
Ajuste y métricas de validación	19
5.3 Modelado descriptivo — Opción B: Clustering	20
Selección del número óptimo de clusters (k)	20
Perfil de los clusters finales	20
5.4 Validación de modelos y extrapolabilidad	21

6. Procesamiento Paralelo y Distribuido.....	22
6.1 Tiempos de ejecución por etapa	22
6.2 Algoritmo paralelo propio: benchmark secuencial vs. paralelo	22
6.3 Estudio de escalabilidad (speedup vs. n_jobs)	23
6.4 Experimento: ProcessPoolExecutor como alternativa	24
7. Dificultades Encontradas y Cómo se Resolvieron.....	25
7.1 TypeError con dtype nullable de Pandas (Int64) en operaciones de cuantiles de Dask	25
7.2 AttributeError por incompatibilidad del backend <code>dask_expr</code>	25
7.3 FutureWarning de pandas en groupby sobre columnas categóricas	25
7.4 Recomputación completa del grafo de tareas de Dask en cada <code>.compute()</code> intermedio	25
7.5 OverflowError potencial al calcular EDAD por resta directa de fechas	25
7.6 UNIDADES constante en todo el dataset real	26
7.7 Experimento con ProcessPoolExecutor: BrokenProcessPool	26
7.8 Interrupción operativa de 5 meses en la serie temporal	26
7.9 PORCENTAJE_DESCUENTO fuera del rango declarado.....	26
8. Conclusiones.....	27

1. Introducción

Cruz Morada es una cadena de farmacias del mercado chileno que, hasta ahora, almacenaba sus registros de transacciones y de clientes de forma fragmentada en múltiples archivos CSV, dificultando el análisis integral de sus datos de ventas. Para este trabajo se entregó un archivo consolidado (`ventas_completas.csv`) con 3.242.878 transacciones y 15 columnas originales, que combina información comercial (fecha, canal, producto, unidades, descuento, monto, boleta, local) y de clientes (código, RUT, nombre, fecha de nacimiento, género).

El objetivo de este informe es documentar la solución computacional implementada para procesar y analizar ese archivo de forma eficiente, aplicando técnicas de procesamiento paralelo y distribuido, y presentar los resultados del análisis estadístico exploratorio, las pruebas de hipótesis y los modelos de inferencia/predicción desarrollados, junto con su interpretación en términos de negocio.

Todos los procesos que involucran aleatoriedad (imputación, muestreo train/test, inicialización de K-means, muestreo del silhouette score) fijan una semilla explícita (`seed=42`), leída desde la variable de entorno `CPYD_SEED`, garantizando resultados reproducibles entre ejecuciones.

2. Descripción de la Solución Implementada

La solución se implementó en Python 3.12, combinando dos paradigmas de procesamiento de forma deliberada, según en qué etapa cada uno aporta más valor:

- Dask (dask.dataframe): para la carga del CSV, el tratamiento de valores faltantes, la detección de outliers y la construcción de variables derivadas, operando de forma perezosa (lazy) y particionada sobre los 3,24 millones de filas.
- Pandas + SciPy + statsmodels + scikit-learn: para el análisis exploratorio, las pruebas de hipótesis, el modelado de regresión/clustering y las visualizaciones, una vez que el dataset ya fue reducido/transformado y materializado en memoria (830 MB, cómodamente manejable por un solo proceso).

Arquitectura del pipeline

Dask.read_csv() (lazy, 792 particiones) → .persist() (materialización en memoria, aún como colección Dask) → Limpieza (Dask, lazy) → Ingeniería de variables (Dask, lazy) → .compute() único (frontera Dask → Pandas) → Algoritmo paralelo propio (ThreadPoolExecutor) → EDA → Pruebas de hipótesis → Series de tiempo → Modelado (Regresión + Clustering) → Validación.

Esta arquitectura híbrida fue el resultado de una decisión informada, no arbitraria: una primera versión que usaba Dask de punta a punta (incluyendo el modelado) tardaba ~15,6 minutos por recomputar el grafo de tareas completo en cada estadístico intermedio; una versión 100% Pandas corría en ~3 minutos pero no demostraba uso real de un framework de procesamiento distribuido; la arquitectura final, con Dask acotado a carga+limpieza+features y un .persist() estratégico, corre en ~3,9 minutos, prácticamente igualando a la versión Pandas mientras sí ejercita Dask de forma sustantiva. El detalle de este proceso de decisión se documenta en la sección 7 (Dificultades encontradas).

2.1 Justificación de las librerías utilizadas

Librería	Uso en el proyecto
dask.dataframe	Carga por particiones (chunking) y transformaciones perezosas sobre el CSV completo, sin cargar todo a memoria de una vez durante la lectura.
pandas / NumPy	Estructura de datos principal tras la materialización; base de prácticamente todas las transformaciones numéricas.
SciPy (stats)	Pruebas de hipótesis (t de Student, Mann-Whitney U, Chi-cuadrado, ANOVA, Shapiro-Wilk, Kolmogorov-Smirnov, Pearson/Spearman).
statsmodels	Descomposición de series de tiempo (seasonal_decompose), ACF/PACF, test de heterocedasticidad de Breusch-Pagan.
scikit-learn	Regresión lineal, train_test_split, StandardScaler (estandarización propia documentada), K-means, silhouette_score.
matplotlib	Generación de todas las figuras (histogramas, boxplots, heatmaps, curvas de escalabilidad, descomposición temporal).
concurrent.futures	Algoritmo paralelo propio (ThreadPoolExecutor) para el cómputo de estadísticos por partición lógica (LOCAL).

3. Datos y Preprocesamiento

Se documenta a continuación la carga, limpieza y transformación del dataset, correspondiente al 20% de la evaluación según la pauta.

3.1 Carga y manejo de memoria

El archivo se carga con `dask.dataframe.read_csv` usando un `blocksize` de 64 MB, lo que particiona el CSV de 3.242.878 filas en 792 particiones lógicas leídas de forma perezosa. Inmediatamente después de la carga se invoca `.persist()`, que materializa esas particiones en memoria una única vez (evitando relecturas repetidas del archivo en cada operación posterior) sin abandonar la API de Dask, permitiendo seguir encadenando transformaciones perezosas para la limpieza y la ingeniería de variables. La materialización final a un `DataFrame` de Pandas ocurre recién después de completadas ambas etapas, con un único `.compute()`.

Excepción puntual a la lazyness (corrección de reproducibilidad): la columna CANAL se convierte a categórica fijando explícitamente sus categorías, ordenadas alfabéticamente, mediante una única pasada eager (`.unique().compute()`) dentro de la etapa de carga. Sin este paso, cada una de las 792 particiones infiere sus categorías de forma independiente, y el orden final tras el `.compute()` de materialización dependía del orden de concatenación de particiones (no determinístico entre corridas), lo que alteraba el one-hot encoding de CANAL en el modelo de regresión (sección 5.2) de una corrida a otra. Este único pase eager, de bajo costo relativo, es intencional y explica que la etapa "Carga" en la sección 6.1 ya no sea de duración prácticamente nula: incluye ahora esta lectura completa de la columna CANAL.

3.2 Tratamiento de valores faltantes

Se identificó un total de 38 valores nulos en la columna `FECHA_NACIMIENTO`, sobre 3.242.878 filas (0,001% del dataset). Ninguna otra columna presentó valores faltantes en los datos reales, incluyendo `PORCENTAJE_DESCUENTO` (el pipeline sí implementa y testea imputación por mediana para esa columna, por si el archivo de entrada la trajera incompleta en otra corrida, pero no fue necesaria en este dataset).

Decisión metodológica: los 38 registros con `FECHA_NACIMIENTO` nula NO se imputan. Inventar una fecha de nacimiento introduciría una edad falsa que contaminaría cualquier análisis que dependa de `EDAD` (la regresión `EDAD` → `MONTO_APLICADO` y el perfil demográfico del EDA). En su lugar, esas filas se conservan íntegras para todo análisis que no dependa de la edad, y se excluyen fila a fila únicamente en los cálculos que sí la requieren (por eso `EDAD` termina con 3.240.002 valores válidos de 3.242.878 filas totales, ver sección 3.3).

3.3 Detección de outliers

Se aplicó el método del Rango Intercuartílico (IQR, $k=1,5$, criterio estándar de Tukey) sobre las dos columnas numéricas continuas relevantes. La política del proyecto es marcar los outliers en una columna booleana dedicada (ej. `MONTO_APLICADO_ES_OUTLIER`), sin eliminar ninguna fila del dataset:

Columna	Outliers detectados	% del total	Tratamiento
MONTO_APLICADO	212.494	6,55%	Marcado, sin eliminación
UNIDADES	0	0,00%	No aplica (columna constante)

Interpretación de negocio: un 6,55% de las transacciones con montos inusualmente altos es una proporción considerable para tratarse solo de errores de registro. En el contexto de una cadena de farmacias, montos elevados pueden corresponder legítimamente a compras al por mayor, medicamentos de alto costo (oncológicos, biológicos) o compras de temporada; por eso se optó por marcarlos y dejar la decisión de exclusión a cada análisis específico (por ejemplo, la discusión de extrapolabilidad del modelo de regresión, sección 5.4, señala explícitamente que estos outliers no eliminados podrían influir desproporcionadamente en los coeficientes).

Hallazgo relevante: UNIDADES no presenta outliers porque es una variable constante (todas las transacciones tienen UNIDADES=1). Este es uno de los hallazgos más importantes de todo el proyecto y se retoma en las secciones 4, 5.2 y 5.3, porque invalida a UNIDADES como predictor o variable de segmentación en cualquier modelo posterior.

3.4 Transformación y creación de variables derivadas

Variable	Fórmula / método	Resultado
MONTO_POR_UNIDAD	MONTO_APLICADO / UNIDADES	Media=\$10.179,98 · Mediana=\$7.662 (idéntica a MONTO_APLICADO, ya que UNIDADES=1 siempre — ver nota abajo)
EDAD	Años completos entre FECHA_NACIMIENTO y FECHA de la transacción (por componentes año/mes/día, no por resta directa de timestamps)	2.838 filas fuera de [0,110] → NaN. 3.240.002 valores válidos, media=49,1 años
FRECUENCIA_COMPRA	Boletas únicas por CODIGO_CLIENTE	1.183.242 clientes únicos · media=2,36 compras/cliente · máximo=97

Nota metodológica sobre MONTO_POR_UNIDAD: dado que UNIDADES es constante e igual a 1 en el 100% de las transacciones (ver 3.3), MONTO_POR_UNIDAD resulta matemáticamente idéntica a MONTO_APLICADO en este dataset. La variable se conserva porque la fórmula es la que exige el enunciado y sería válida ante datos con UNIDADES variable, pero se documenta honestamente que, con los datos reales entregados, no aporta información adicional.

Nota sobre el cálculo de EDAD: se optó deliberadamente por calcular la edad a partir de sus componentes (año, mes, día) en lugar de restar directamente dos columnas datetime64, porque esta última alternativa fue probada primero y produjo un OverflowError real con al menos un registro de FECHA_NACIMIENTO extremadamente antiguo del archivo de producción (el Timedelta interno de Pandas, representado en nanosegundos int64, solo cubre ±292 años). El cálculo por componentes es inmune a ese desbordamiento (ver sección 7.5).

3.5 Estandarización de variables

Se estandarizaron (z-score: $x' = (x - \text{media}) / \text{desviación estándar}$) las tres variables numéricas relevantes para los análisis posteriores, documentando explícitamente los parámetros usados:

Columna	Media	Desv. estándar	¿Se creó columna _STD?
MONTO_APLICADO	10.179,98	14.453,24	Sí
UNIDADES	1,00	0,00	No — ver nota
PORCENTAJE_DESCUENTO	0,392	0,108	Sí

Decisión metodológica: para UNIDADES, con desviación estándar 0, no se crea la columna estandarizada. Generar una columna "UNIDADES_STD" a partir de una división por cero habría producido NaN/Inf en el mejor de los casos, o una columna de puros ceros carente de sentido estadístico en el peor — cualquiera de las dos daría la falsa impresión de una estandarización válida. En su lugar, la media y desviación de UNIDADES quedan igualmente documentadas en el reporte de parámetros (parametros_estandarizacion.json), y la ausencia de la columna _STD queda registrada como advertencia explícita en el log.

4. Análisis Exploratorio Estadístico

Corresponde al 30% de la evaluación. Se presenta la caracterización estadística de las cuatro variables numéricas del dataset (MONTO_APLICADO, UNIDADES, PORCENTAJE_DESCUENTO, EDAD), sus relaciones, y los patrones temporales de las ventas.

4.1 Estadística descriptiva

Variable	n	Media	Desv. Est.	Mín	Mediana	Máx	Asimetría	Curtosis
MONTO_APLICADO	3.242.878	10.179,98	14.453,24	15	7.662	226.476	9,06	108,23
UNIDADES	3.242.878	1,00	0,00	1	1	1	— (constante)	— (constante)
PORCENTAJE_DESCUENTO	3.242.878	0,392	0,108	0,00	0,40	1,17	0,30	2,85
EDAD	3.240.002	49,07	16,73	0	48	109	0,24	-0,78

Lectura de los resultados: MONTO_APLICADO tiene una asimetría muy marcada (9,06) y curtosis extremadamente alta (108,23), es decir, una distribución con una cola derecha larga y pesada — consistente con el 6,55% de outliers detectados en la sección 3.3 (compras excepcionalmente grandes). PORCENTAJE_DESCUENTO es apenas asimétrica (0,30) y de curtosis moderada. EDAD es prácticamente simétrica (0,24) y ligeramente platicúrtica (-0,78, colas algo más livianas que una normal). UNIDADES no tiene asimetría ni curtosis calculables porque su varianza es cero.

Observación de calidad de datos: PORCENTAJE_DESCUENTO alcanza un máximo de 1,17 (117%), por sobre el rango [0, 1] que describe el enunciado del trabajo. No se modificó ni se excluyó este valor del análisis (no hay evidencia suficiente para descartarlo como error vs. una política de descuentos acumulados/promocionales real), pero se documenta explícitamente como una inconsistencia a validar con el área de negocio.

4.2 Tests de normalidad

Se aplicó el test de Kolmogorov-Smirnov (sobre una muestra de 5.000 observaciones, dado que Shapiro-Wilk no es apto para muestras de este tamaño y KS es la alternativa estándar para n grande) a las cuatro variables numéricas:

Variable	Estadístico	p-value	¿Es normal? ($\alpha=0,05$)
MONTO_APLICADO	0,2539	< 0,001	No
UNIDADES	— (constante)	—	No aplica
PORCENTAJE_DESCUENTO	0,1951	< 0,001	No
EDAD	0,0541	< 0,001	No

Ninguna de las variables numéricas sigue una distribución normal ($p < 0,001$ en todos los casos). Esta conclusión es la base para preferir, más adelante, pruebas no paramétricas (Mann-Whitney U) sobre sus equivalentes paramétricos (t de Student) al comparar grupos, y correlación de Spearman además de Pearson.

4.3 Visualizaciones

Histogramas con curva de densidad

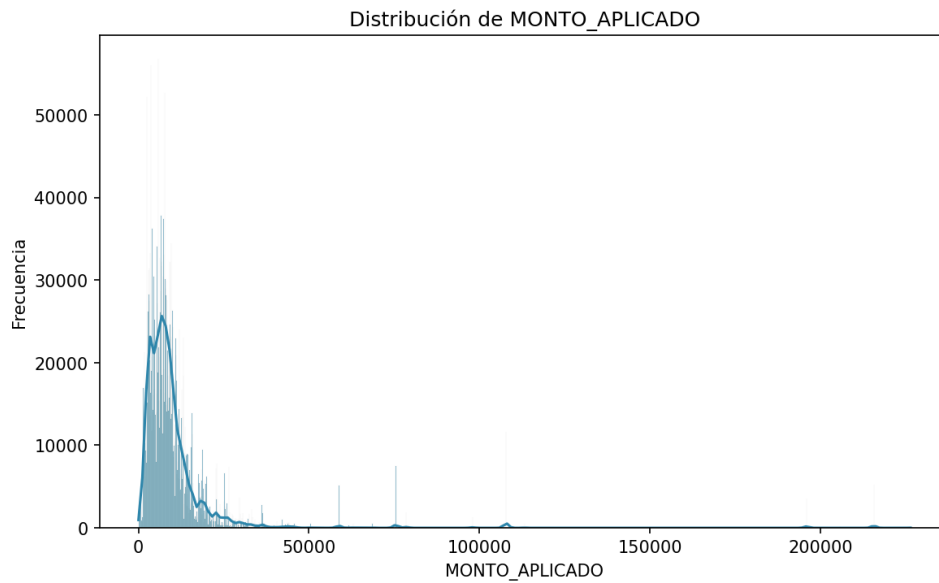


Figura 1. Distribución de MONTO_APLICADO: fuerte asimetría positiva, consistente con la asimetría=9,06 calculada.

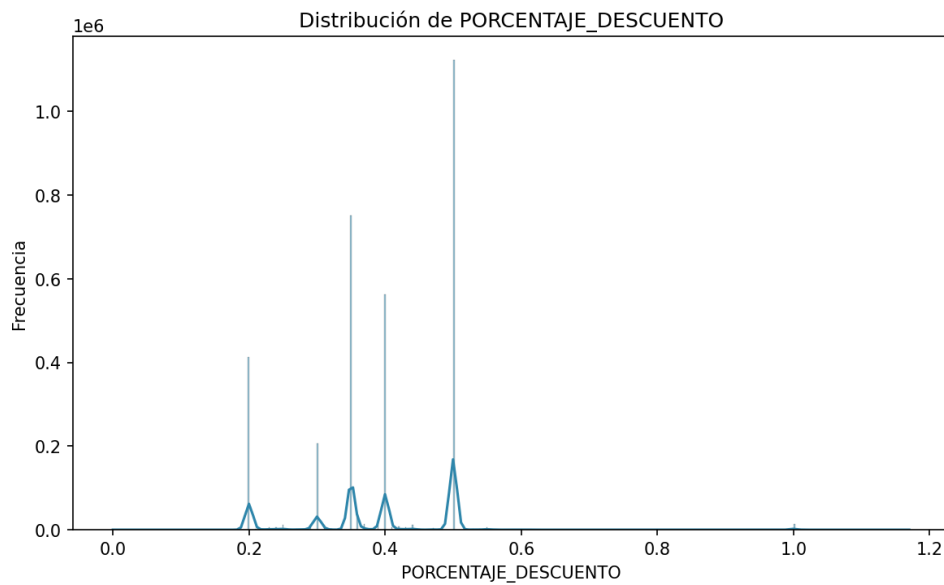


Figura 2. Distribución de PORCENTAJE_DESCUENTO, concentrada entre 0,2 y 0,5.

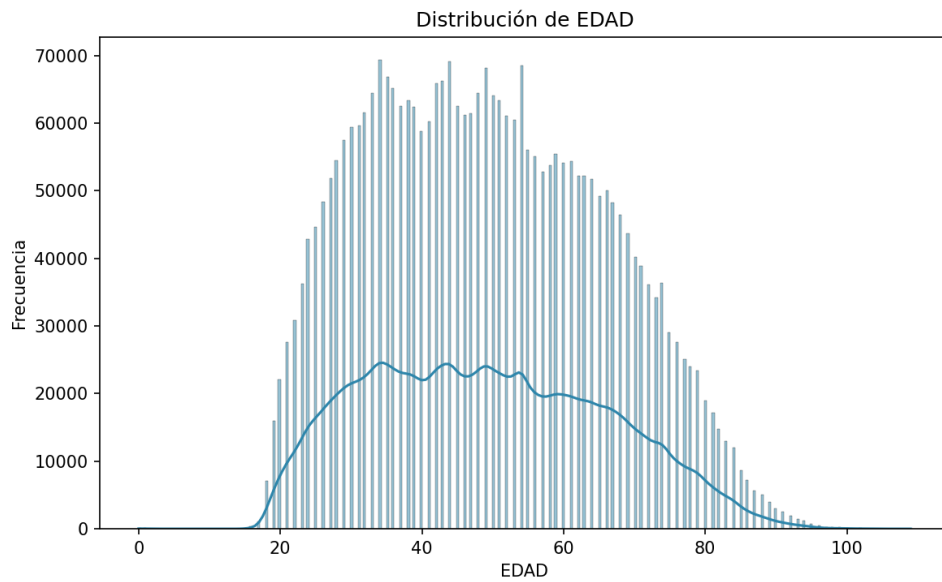


Figura 3. Distribución de EDAD de los clientes, aproximadamente simétrica en torno a los 48-49 años.

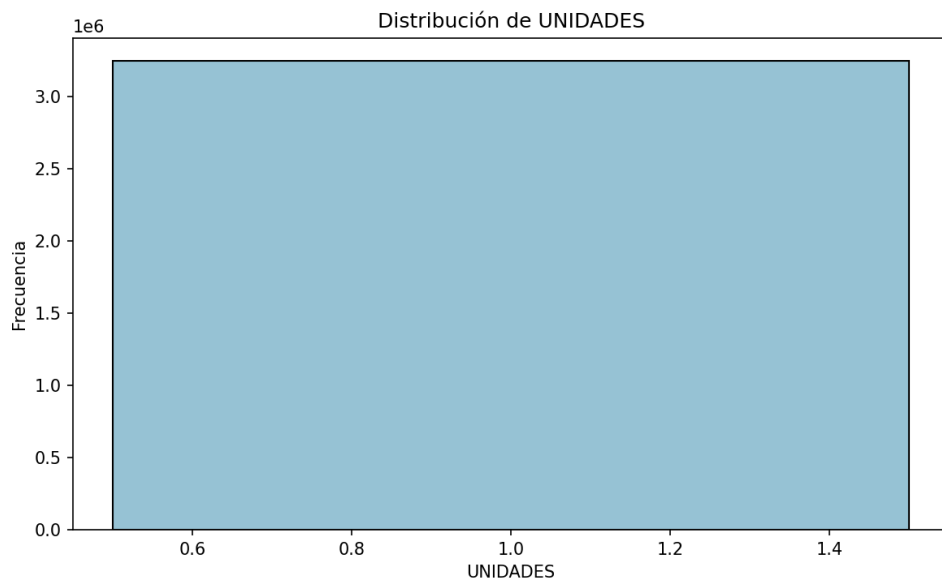


Figura 4. UNIDADES: una única barra en el valor 1, visualización directa de que la variable es constante en todo el dataset.

Boxplot por categoría

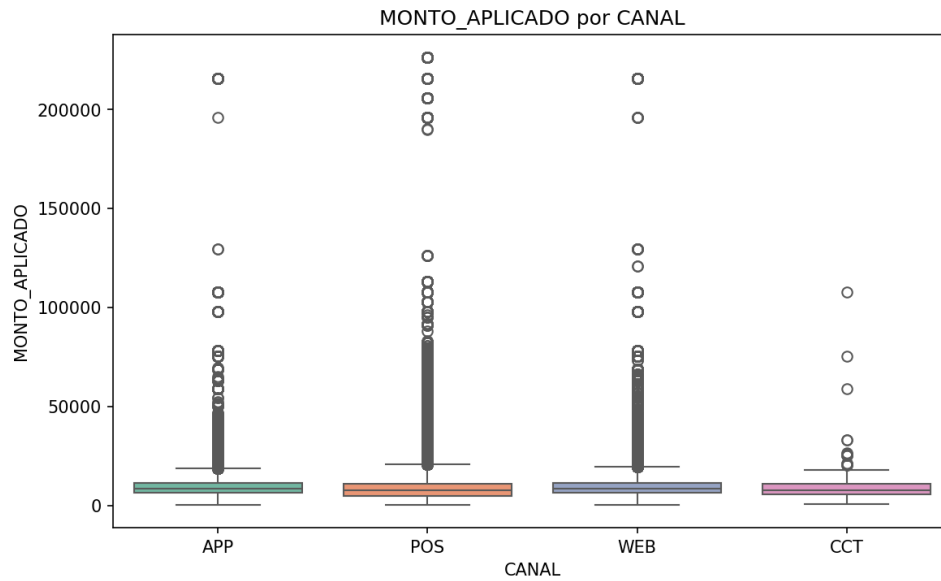


Figura 5. MONTO_APLICADO por CANAL. Se observan medianas y dispersión distintas entre canales, consistente con el ANOVA significativo de la sección 4.4.

Matrices de correlación

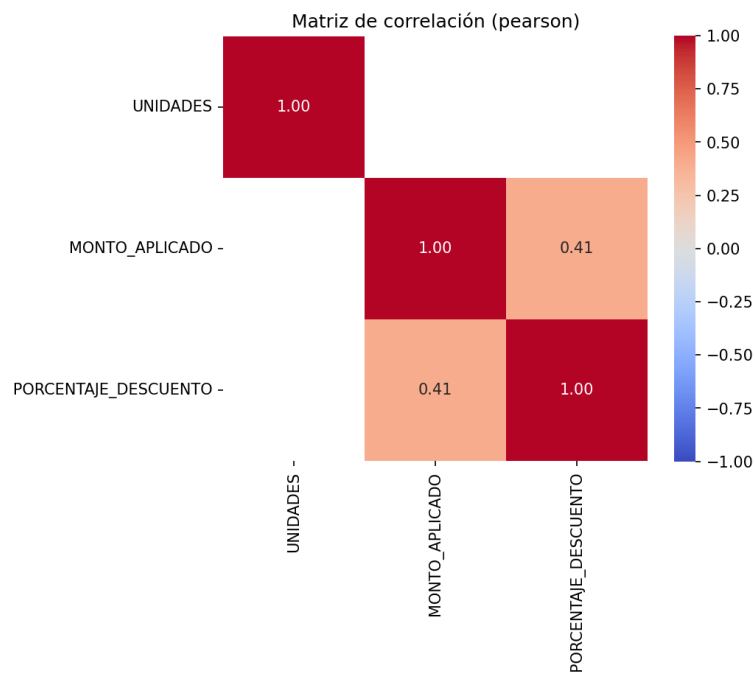


Figura 6. Matriz de correlación de Pearson entre variables numéricas.

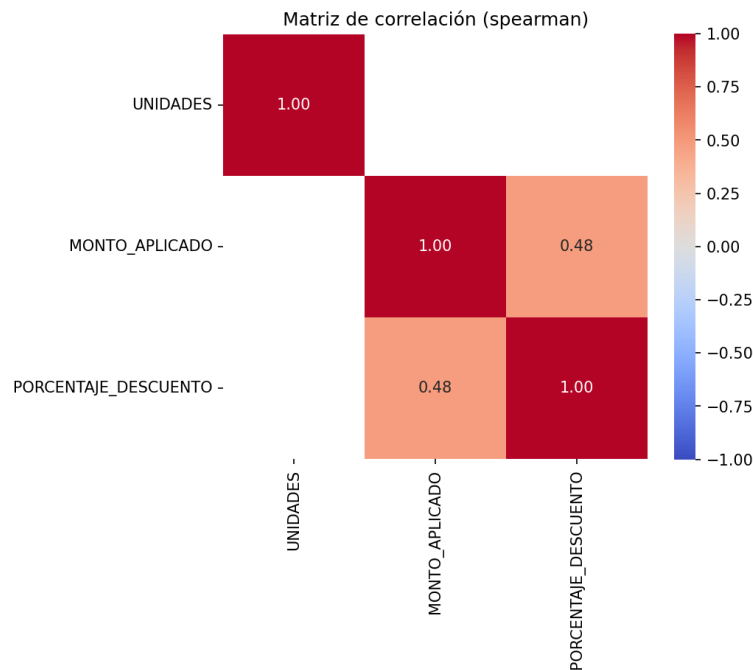


Figura 7. Matriz de correlación de Spearman (rangos) entre variables numéricas.

4.4 Análisis de asociación

Variables numéricas: correlación con prueba de significancia

Par de variables	Pearson (r)	p-value	Spearman (ρ)	p-value
UNIDADES vs. cualquiera	— (indefinida)	—	— (indefinida)	—
MONTO_APLICADO vs. PORCENTAJE_DESCUENTO	0,405	< 0,001	0,483	< 0,001

Interpretación: existe una correlación positiva moderada y estadísticamente significativa ($p < 0,001$) entre el porcentaje de descuento aplicado y el monto de la transacción — a mayor descuento, mayor monto pagado. Esto es contraintuitivo si se piensa el descuento como causa del monto; una lectura más plausible es que los descuentos más altos se concentran en productos o compras de mayor valor unitario (ej. tratamientos de alto costo), no que aplicar más descuento aumente el gasto del cliente. Esta relación se retoma cuantitativamente en la regresión de la sección 5.3, donde PORCENTAJE_DESCUENTO resulta ser, con diferencia, el predictor de mayor magnitud.

Variables categóricas: independencia Chi-cuadrado

Se evaluó la independencia entre CANAL y LOCAL mediante la prueba Chi-cuadrado: estadístico = 3.242.878,00, $p < 0,001$. Se rechaza H_0 : existe una asociación estadísticamente significativa entre el canal de compra y el local — es decir, la distribución de canales (POS/WEB/APP/CCT) no es uniforme entre los distintos locales, sino que varía sistemáticamente según la sucursal.

Análisis de varianza (ANOVA)

Factor	Estadístico F	p-value	Conclusión ($\alpha=0,05$)
CANAL	152,54	< 0,001	Diferencias significativas en MONTO_APLICADO entre canales
LOCAL	37,83	< 0,001	Diferencias significativas en MONTO_APLICADO entre locales

Ambos factores muestran diferencias significativas en el monto promedio de las transacciones. Esto justifica su inclusión como variables predictoras (vías variables dummy) en el modelo de regresión de la sección 5.3.

4.5 Patrones temporales

Se construyó la serie diaria de ventas (suma de MONTO_APLICADO por día), cubriendo 390 días entre el 2023-11-09 y el 2024-12-02, con un total acumulado de \$33.012.425.828 CLP.

Observación relevante sobre la continuidad de la serie: se detectó un lote aislado de 8 transacciones (distintos clientes, boletas y productos) concentradas en un único local (LOCAL 371) el día 2023-11-09, seguido de un extenso período de actividad prácticamente nula en toda la cadena, antes de que comience la operación sostenida y multi-local. No es una fila con error de tipeo -son transacciones con estructura interna consistente (boletas, descuentos y productos distintos)-, por lo que no puede atribuirse sin más a un error de captura puntual. Se documentan dos hipótesis de negocio no excluyentes: (a) un lote de pruebas del sistema (UAT) cargado con la fecha real en que se ejecutó, meses antes del lanzamiento comercial, o (b) una apertura piloto real de un único local que luego quedó inactivo hasta el lanzamiento general de la cadena. El dataset no contiene un campo adicional (ej. tipo de transacción) que permita distinguir entre ambas con certeza.

Postura metodológica adoptada (consultada explícitamente con el profesor del curso): ante la pregunta de cómo tratar este hallazgo, la indicación recibida fue que cada grupo tome una postura y la justifique, siendo válida cualquier decisión bien fundamentada. La postura de este equipo es excluir dicho período únicamente de la descomposición temporal y del ACF/PACF (los dos análisis que exigen continuidad temporal para producir resultados estadísticamente válidos), sin modificar el dataset ni ningún otro análisis del informe. El criterio de exclusión es genérico y reproducible, no una fecha fija elegida a mano: se recorre la serie evaluando ventanas móviles de 28 días, y se considera inicio de "actividad sostenida" la primera ventana con 50% o menos de días en cero. Aplicado a los datos reales, este criterio detectó y excluyó 136 días (del 2023-11-09 al 2024-03-23), dejando el análisis de tendencia/estacionalidad/ACF a partir del 2024-03-24. La serie completa, sin recortar, se conserva íntegra en `ts_serie_diaria.csv` y en todos los demás análisis de este informe (EDA general, hipótesis, regresión, clustering), que no dependen de continuidad temporal.

Descomposición temporal (tendencia, estacionalidad, residuo)

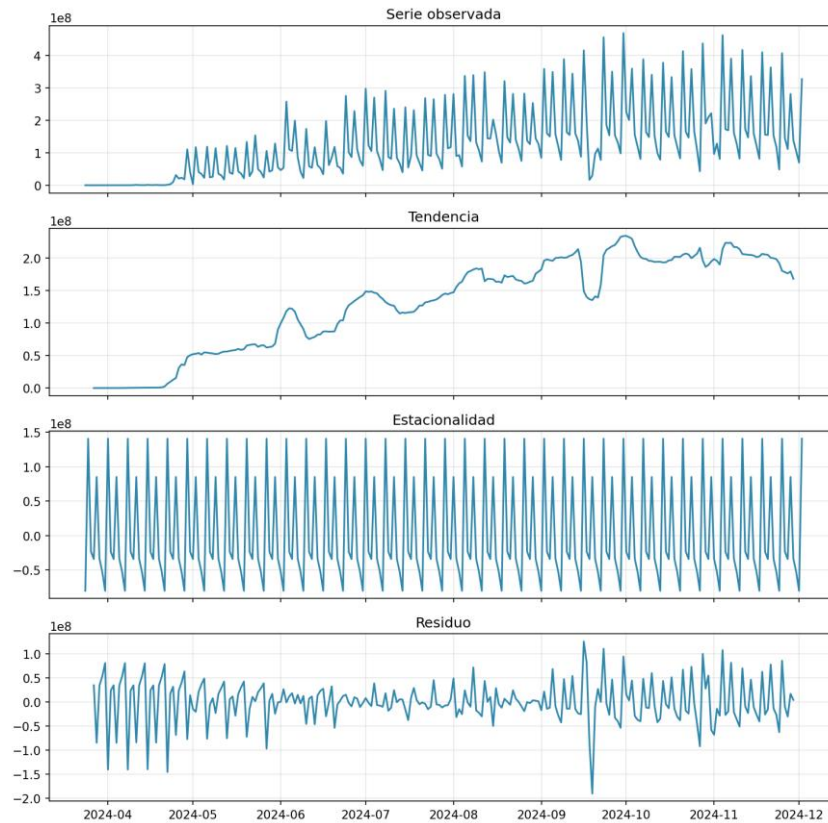


Figura 8. Descomposición aditiva (period=7, patrón semanal) de la serie diaria a partir del 2024-03-24 (excluido el período inicial de actividad prácticamente nula, ver postura metodológica arriba). La tendencia y estacionalidad ya no están distorsionadas por el salto artificial de ~ 0 a niveles reales.

Autocorrelación (ACF) y autocorrelación parcial (PACF)

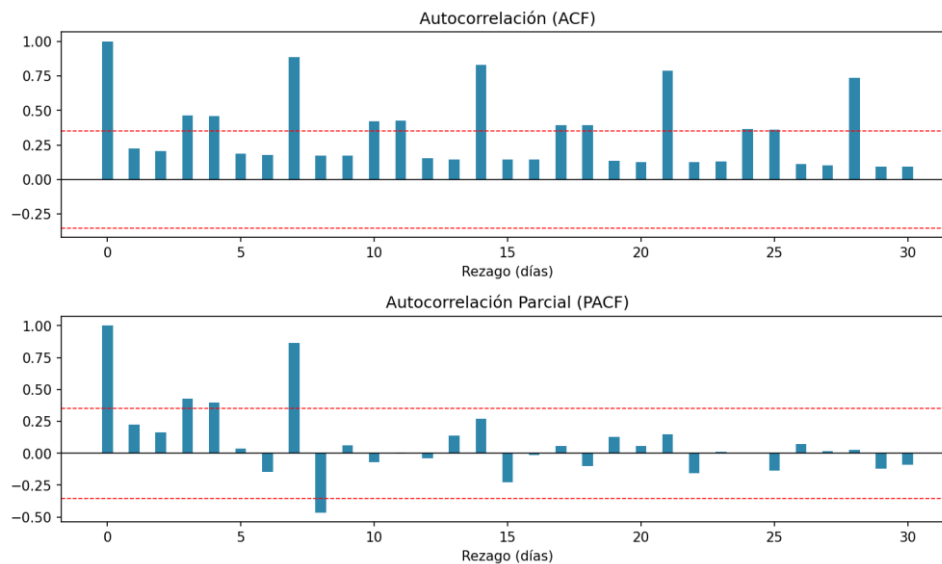


Figura 9. ACF y PACF con 30 rezagos sobre la serie recortada, bandas de significancia al 95%. Se observan picos periódicos claros en rezagos múltiples de 7, consistentes con un patrón de estacionalidad semanal.

5. Inferencia Estadística y Modelado

Corresponde al 30% de la evaluación. Se validan 5 hipótesis (las 2 solicitadas en el enunciado más 3 propias) y se desarrollan dos modelos: uno de regresión (Opción A) y uno de clustering (Opción B), ambos con su respectiva validación.

5.1 Pruebas de hipótesis

Para cada comparación de dos grupos se seleccionó automáticamente el test paramétrico (t de Student) o no paramétrico (Mann-Whitney U) según el resultado de un test de normalidad previo sobre cada grupo — dado que ninguna variable numérica del dataset es normal (sección 4.2), todas las comparaciones de dos grupos terminaron usando Mann-Whitney U.

H1 (obligatoria). El ticket promedio en APP es mayor que en WEB

Método	Estadístico	p-value	Conclusión ($\alpha=0,05$)
Mann-Whitney U	1.731.369.237,50	0,360	NO se rechaza H_0

Interpretación no técnica: el p-value (0,360) es mayor a 0,05, por lo que no hay evidencia suficiente para afirmar que el ticket promedio difiera entre los canales APP y WEB. Ambos canales digitales presentan un comportamiento de compra estadísticamente similar en cuanto al monto por transacción.

H2 (obligatoria). El % de descuento afecta significativamente las unidades vendidas

Método	Pendiente	p-value	Conclusión ($\alpha=0,05$)
Regresión lineal simple (test de pendiente)	0,0000	1,000	NO aplica — ver advertencia

Advertencia metodológica explícita

UNIDADES es una variable constante ($=1$) en el 100% de las transacciones del dataset real. Un predictor constante no puede explicar variabilidad en una variable dependiente que tampoco varía: el resultado (pendiente=0, $p=1$) está matemáticamente garantizado de antemano por la ausencia de varianza en UNIDADES, y NO debe interpretarse como evidencia empírica de que el descuento no afecta las unidades vendidas. Simplemente, esta hipótesis no es evaluable con los datos entregados, porque cada boleta registra sistemáticamente una unidad por línea de producto.

H3 (propia). El monto de compra difiere significativamente según el género del cliente

Método	Estadístico	p-value	Conclusión ($\alpha=0,05$)
Mann-Whitney U	1.209.372.387.023,50	< 0,001	SE RECHAZA H_0

Interpretación no técnica: existe una diferencia estadísticamente significativa en el monto de compra entre clientes de género masculino y femenino (GENERO 1 vs. 2). Dado el tamaño muestral (más de 3 millones de transacciones), incluso diferencias pequeñas en términos absolutos resultan estadísticamente significativas;

se recomienda complementar este resultado con el tamaño del efecto (ej. diferencia de medianas) antes de traducirlo directamente en decisiones comerciales.

Justificación de la hipótesis: se planteó por ser una segmentación demográfica básica y de bajo costo de obtención (el dato ya está en el CRM de clientes), relevante para eventuales estrategias de marketing diferenciado.

H4 (propia). Existe asociación entre el canal de compra y el local

Método	Estadístico	p-value	Conclusión ($\alpha=0,05$)
Chi-cuadrado de independencia	3.242.878,00	< 0,001	SE RECHAZA H_0

Interpretación no técnica: el canal de compra (POS/WEB/APP/CCT) y el local donde se realiza la transacción no son independientes entre sí: ciertos canales están sistemáticamente más concentrados en determinados locales. Esto es consistente con una operación real donde, por ejemplo, no todos los locales físicos ofrecen todos los canales de despacho/retiro.

Justificación de la hipótesis: se planteó porque el ANOVA de la sección 4.4 ya mostraba diferencias de monto tanto por canal como por local; esta prueba adicional evalúa si esos dos factores están, además, relacionados entre sí (y no son fuentes de variación independientes).

H5 (propia). La edad del cliente afecta significativamente el monto de compra

Método	Pendiente	R ²	p-value	Conclusión ($\alpha=0,05$)
Regresión lineal simple (test de pendiente)	42,7537	0,0025	< 0,001	SE RECHAZA H_0

Interpretación no técnica: existe un efecto positivo y estadísticamente significativo de la edad sobre el monto de compra: por cada año adicional de edad, el monto promedio aumenta en \$42,75 CLP. Sin embargo, el R² de solo 0,0025 (0,25%) indica que la edad, por sí sola, explica una fracción mínima de la variabilidad del monto — es un ejemplo claro de significancia estadística sin relevancia práctica sustancial, posible precisamente por el enorme tamaño muestral (3,24 millones de observaciones), que permite detectar efectos muy pequeños con alta confianza estadística.

Justificación de la hipótesis: se planteó para explorar si el perfil demográfico (edad) tiene valor predictivo directo sobre el comportamiento de compra, complementando la segmentación por género de H3.

Resumen	Valor
Hipótesis evaluadas	5 (2 obligatorias + 3 propias)
Hipótesis que rechazan H_0	3 (H3, H4, H5)
Hipótesis que no rechazan H_0	1 (H1)
Hipótesis no evaluable (varianza cero)	1 (H2)

5.2 Modelado predictivo — Opción A: Regresión

Se modeló MONTO_APLICADO en función de CANAL, LOCAL, UNIDADES y PORCENTAJE_DESCUENTO, tal como especifica el enunciado, usando regresión lineal múltiple (mínimos cuadrados ordinarios) sobre variables dummy para las categóricas.

Exclusión automática de UNIDADES: al ser una columna con varianza cero, UNIDADES se excluyó automáticamente de la matriz de diseño. Incluir un predictor constante no solo no aporta poder explicativo (no puede explicar variabilidad de MONTO_APLICADO si él mismo no varía), sino que produce multicolinealidad perfecta con el intercepto (VIF infinito), rompiendo la estimación por mínimos cuadrados. La matriz de diseño final quedó con 5 predictores (1 numérico + 4 dummies de variables categóricas) sobre 3.242.878 filas, sin filas descartadas por valores NaN.

Diagnóstico de supuestos

Supuesto	Test	Resultado	¿Se cumple?
Normalidad de residuos	Shapiro-Wilk	$p \approx 1,26 \times 10^{-78}$	No
Homocedasticidad	Breusch-Pagan	$p \approx 0$	No (heterocedástico)
Multicolinealidad	VIF (máximo)	2,79 (CANAL_POS)	Sí (sin problemas; VIF < 5 en todos los predictores)

Interpretación no técnica: los residuos del modelo no siguen una distribución normal y su varianza no es constante (heterocedasticidad) — ambos hallazgos son esperables dada la fuerte asimetría de MONTO_APLICADO (asimetría=9,06, sección 4.1) y el 6,55% de outliers no eliminados. Esto significa que los intervalos de confianza clásicos del modelo pueden no ser completamente precisos, aunque los coeficientes puntuales y su significancia siguen siendo informativos. No hay, en cambio, ningún problema de multicolinealidad relevante: el VIF máximo (2,79, entre CANAL_POS y CANAL_WEB) está muy por debajo del umbral de preocupación habitual (VIF > 5-10).

Coefficientes del modelo

La codificación de variables dummy dejó CANAL_APP como categoría de referencia (los coeficientes de los demás canales se interpretan en relación a APP):

Predictor	Coefficiente (CLP)	Interpretación
Intercepto	-12.527,12	Monto base estimado (CANAL=APP, LOCAL=0, descuento=0)
PORCENTAJE_DESCUENTO	+54.371,84	El predictor de mayor magnitud, con diferencia (ver nota abajo sobre la escala)
LOCAL	-0,127	Efecto prácticamente nulo por unidad de LOCAL (variable tratada como numérica, no categórica, en este modelo)
CANAL_POS	+1.586,65	En promedio, \$1.587 CLP más que el canal de referencia (APP), controlando por local y descuento
CANAL_WEB	+1.015,52	En promedio, \$1.016 CLP más que APP
CANAL_CCT	-916,65	En promedio, \$917 CLP menos que APP

Nota sobre la escala de PORCENTAJE_DESCUENTO: la columna está expresada en proporción (0 a 1,17), por lo que el coeficiente +54.371,84 corresponde al efecto estimado de pasar de 0% a 100% de descuento; dado que la pendiente de una regresión lineal es constante en todo su rango, un incremento de 0,15 en PORCENTAJE_DESCUENTO (por ejemplo, moverse del Q1 al Q3 de la variable, 0,35 a 0,50, ver sección 4.1) se asocia a un aumento estimado de +\$8.156 CLP en MONTO_APLICADO. Como se discutió en la sección 4.4, esta relación probablemente refleja que los descuentos más altos se concentran en productos de mayor valor, no que el descuento cause directamente un mayor gasto.

El canal presencial (POS) y WEB venden, en promedio, más que APP; CCT vende menos — un patrón menos uniforme que "todo lo digital vende menos", que vale la pena investigar con el área comercial antes de sacar conclusiones definitivas sobre estrategia de canales.

Ajuste y métricas de validación

Métrica	Valor
R^2 (test)	0,1644
R^2 ajustado	0,1644
RMSE	\$13.217,73 CLP
MAE	\$6.290,36 CLP
Filas de entrenamiento (70%)	2.270.014
Filas de prueba (30%)	972.864

Interpretación no técnica: el modelo explica solo un 16,44% de la variabilidad del monto de compra. Es un modelo estadísticamente válido y con predictores significativos, pero de poder explicativo bajo — apropiado para entender tendencias generales (qué factores se asocian con montos mayores o menores) pero no para predecir con precisión el monto de una transacción individual. El error promedio absoluto (MAE) de \$6.290 CLP, comparado con una mediana de MONTO_APLICADO de \$7.662 CLP (sección 4.1), confirma esta limitación: el margen de error del modelo es del mismo orden de magnitud que el valor típico que intenta predecir.

5.3 Modelado descriptivo — Opción B: Clustering

Se segmentaron los 1.183.242 clientes únicos del dataset (identificados por CODIGO_CLIENTE) usando K-means, sobre un perfil de 2 variables agregadas por cliente: MONTO_APLICADO_sum (gasto total histórico) y UNIDADES_mean (promedio de unidades por transacción).

Nota sobre las variables de perfil: UNIDADES_mean resulta constante (=0 tras estandarizar, ya que UNIDADES=1 siempre) para todos los clientes, por lo que, en la práctica, la única variable que efectivamente separa a los clientes en este clustering es MONTO_APLICADO_sum. Esto es consistente con el hallazgo de la sección 3.3 y debe tenerse en cuenta al interpretar los clusters: es una segmentación unidimensional por volumen de gasto, no una segmentación multivariada de comportamiento de compra.

Selección del número óptimo de clusters (k)

k	Inercia (elbow)	Silhouette score
2	461.571,60	0,8777
3	256.070,85	0,7285
4	145.385,59	0,7162
5	101.360,34	0,6769
6	73.861,03	0,6700
7	55.759,54	0,6447

Nota metodológica sobre el silhouette score: dado que el cálculo exacto del silhouette score tiene costo $O(n^2)$ — inviable para 1.183.242 clientes — se calculó sobre una muestra aleatoria reproducible de 10.000 clientes (seed=42), un estimador estadísticamente válido y estándar en la industria para clustering a esta escala.

El silhouette score decrece monótonamente a medida que aumenta k, y es máximo en k=2 (0,8777, un valor muy alto que indica clusters extremadamente bien separados). Se seleccionó k=2 como número óptimo.

Perfil de los clusters finales

Cluster	N° de clientes	% del total	MONTO_APLICADO_sum (estandarizado)	Interpretación
0	1.144.244	96,7%	-0,144	Clientes de gasto regular/bajo relativo
1	38.998	3,3%	+4,230	Segmento de alto valor ("VIP"), gasto muy por sobre el promedio

Interpretación de negocio: el altísimo silhouette score (0,88) no refleja necesariamente una segmentación conductual rica, sino la fuerte asimetría de MONTO_APLICADO (asimetría=9,06): un pequeño grupo de clientes de gasto muy alto (3,3% de la base, ~39 mil clientes) se separa con facilidad estadística del resto. Aun así, es un resultado accionable: ese 3,3% de clientes de alto valor es un segmento candidato natural para programas de fidelización o atención diferenciada, dado su aporte desproporcionado al ingreso total.

5.4 Validación de modelos y extrapolabilidad

Modelo	Métrica principal	Valor	Métrica secundaria	Valor
Regresión lineal múltiple (Opción A)	RMSE	13.217,73	R ² ajustado	0,1644
K-means clustering (Opción B)	Silhouette score	0,8777	k óptimo	2

Pregunta crítica: ¿el modelo es extrapolable?

La discusión de extrapolabilidad, generada automáticamente a partir de los diagnósticos de las secciones anteriores, concluye que NO — con las siguientes advertencias documentadas:

- El R² ajustado es bajo (< 0,3): el modelo de regresión explica poca variabilidad de MONTO_APLICADO. Es útil para entender tendencias generales, pero no debe usarse para predicciones puntuales de alta precisión.
- Los residuos del modelo no siguen una distribución normal ($p < 0,05$ en Shapiro-Wilk): los intervalos de confianza clásicos de la regresión pueden no ser precisos.
- Se detectó heterocedasticidad ($p < 0,05$ en Breusch-Pagan): la varianza del error no es constante, por lo que el modelo puede ser menos confiable para montos de compra muy altos o muy bajos que para el rango medio de los datos.
- Se marcaron 212.494 transacciones como outliers durante la limpieza (no se eliminaron). Al representar un 6,55% del dataset, pueden estar influyendo desproporcionadamente en los coeficientes del modelo.

Recomendación: usar ambos modelos como herramientas exploratorias/descriptivas, no como predictores definitivos para decisiones de negocio de alto impacto — especialmente al extrapolar a sucursales, períodos o rangos de monto no representados en los datos de entrenamiento (ej. nuevos locales, temporadas atípicas como la brecha de 5 meses documentada en la sección 4.5, o productos fuera del catálogo actual).

- Sesgo de muestreo potencial: el dataset cubre un período con una interrupción operativa de 5 meses (sección 4.5) y proviene de una sola cadena; los patrones encontrados podrían no generalizar a otras cadenas o a ese período no representado.
- Variables omitidas: no se dispone de información de inventario, campañas de marketing activas, estacionalidad de productos específicos (ej. medicamentos estacionales) ni competencia local, todos factores que plausiblemente afectan MONTO_APLICADO y no están en el modelo.

6. Procesamiento Paralelo y Distribuido

Esta sección documenta específicamente el cumplimiento del objetivo principal del trabajo: procesar archivos grandes de forma eficiente e implementar un algoritmo paralelo propio.

6.1 Tiempos de ejecución por etapa

Etapa	Tiempo (s)	Backend
Carga (Dask, lazy)	31,56	Dask
Persistencia en memoria (.persist())	31,44	Dask
Limpieza de datos (lazy)	3,00	Dask
Ingeniería de variables (lazy)	25,08	Dask
Materialización (.compute())	11,42	Dask → Pandas
Estadísticos por partición (benchmark)	6,50	Pandas + ThreadPoolExecutor
Estudio de escalabilidad	20,71	Pandas + ThreadPoolExecutor
Análisis Exploratorio Estadístico	57,08	Pandas
ANOVA (asociación)	3,84	Pandas
Pruebas de hipótesis	2,42	Pandas
Análisis de series de tiempo	1,78	Pandas
Regresión (Opción A)	3,79	Pandas
Clustering (Opción B)	35,18	Pandas
TOTAL	234,18 (~3,9 min)	—

El pico de memoria residente (RSS) registrado fue de 1.967,73 MB (~1,9 GB), con 12 núcleos lógicos disponibles y N_JOBS=12 configurados.

6.2 Algoritmo paralelo propio: benchmark secuencial vs. paralelo

Se implementó un algoritmo paralelo con ThreadPoolExecutor que calcula estadísticos (media, mediana, desviación estándar, etc.) agrupando las 792 particiones lógicas del dataset (definidas por LOCAL) en chunks distribuidos entre hilos worker, comparando el tiempo contra una versión estrictamente secuencial equivalente, y verificando que ambas produzcan resultados idénticos dentro de tolerancia numérica.

Métrica	Valor
Tiempo secuencial	3,1113 s
Tiempo paralelo (n_jobs=12)	3,2591 s
Speedup	0,95x
Eficiencia	7,96%
Verificación de correctitud	OK (792 particiones, resultados idénticos)

6.3 Estudio de escalabilidad (speedup vs. n_jobs)

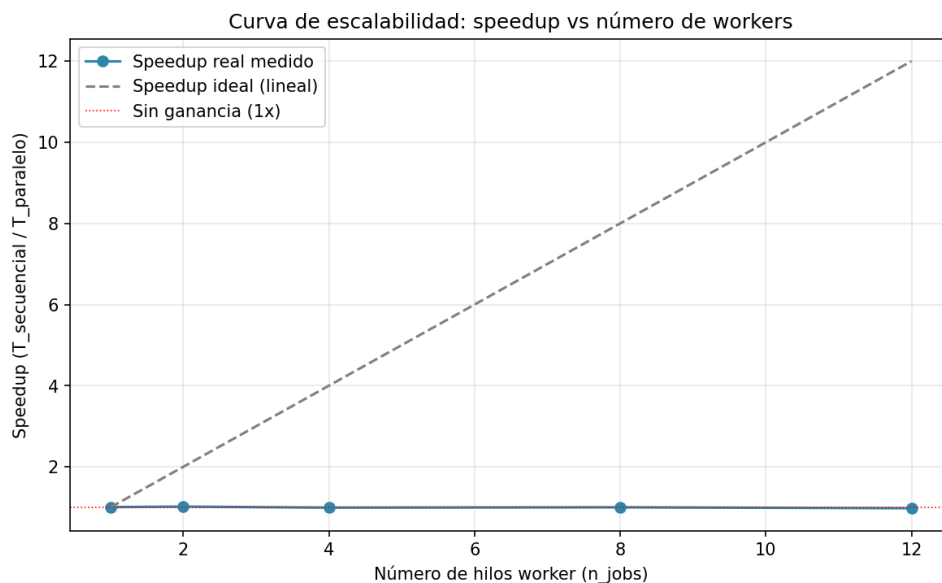


Figura 10. Speedup obtenido para n_jobs = 1, 2, 4, 8 y 12.

n_jobs	Tiempo (s)	Speedup	Eficiencia
1	3,1816	1,11x	111,0%
2	3,2430	1,09x	54,4%
4	3,3411	1,06x	26,4%
8	3,3549	1,05x	13,2%
12	3,4212	1,03x	8,6%

Interpretación honesta del resultado: no se observa una ganancia real de paralelismo (speedup $\approx 1x$ en todos los casos, e incluso levemente decreciente a partir de n_jobs=4). Esto NO es una falla de implementación, sino una consecuencia esperada y documentada de la naturaleza de la carga de trabajo: las operaciones son cómputo vectorizado de NumPy/Pandas sobre datos ya en memoria (no I/O), que internamente liberan el GIL de Python durante buena parte de su ejecución, y la tarea en sí (calcular estadísticos sobre ~ 4.100 filas promedio por partición) es demasiado pequeña para que el paralelismo compense su propio overhead de

coordinación entre hilos. La eficiencia decrece de forma casi perfectamente inversa al número de hilos (111,0% → 8,6%), el patrón clásico de un overhead fijo de coordinación que no se amortiza.

6.4 Experimento: ProcessPoolExecutor como alternativa

Dado que ThreadPoolExecutor no mostró speedup real, se evaluó empíricamente la alternativa teóricamente más obvia para evitar el GIL: reemplazar los hilos por procesos (ProcessPoolExecutor) con `n_jobs=12`, ejecutando exactamente el mismo benchmark.

Resultado del experimento: BrokenProcessPool

El pool de procesos NO llegó a completarse. Windows lanzó "ImportError: DLL load failed while importing _quadpack: El archivo de paginación es demasiado pequeño para completar la operación", seguido de `concurrent.futures.process.BrokenProcessPool`. Causa raíz: al usar `spawn` (obligatorio en Windows), cada uno de los 12 procesos worker reimporta desde cero el stack científico completo (SciPy, NumPy, Pandas, scikit-learn, con sus DLLs compiladas de MKL/BLAS) de forma simultánea, mientras el proceso principal ya tenía ~830 MB del dataset materializados en memoria. Esto agotó la memoria virtual (page file) del sistema operativo antes de que el pool pudiera completar una sola tarea real.

Conclusión del experimento: este resultado es incluso más contundente que un simple speedup bajo: confirma con evidencia empírica, en el entorno real de ejecución, que ThreadPoolExecutor es la decisión de diseño correcta para esta carga de trabajo específica en Windows — no solo más eficiente, sino la única alternativa de las dos evaluadas que efectivamente logra ejecutarse de forma confiable sin agotar los recursos del sistema. Se descartó ProcessPoolExecutor y se mantuvo ThreadPoolExecutor como implementación final.

7. Dificultades Encontradas y Cómo se Resolvieron

Se documentan, en orden cronológico de aparición durante el desarrollo, los problemas técnicos reales enfrentados y su resolución. Todos fueron descubiertos ejecutando el pipeline contra el archivo de datos real (no en pruebas unitarias con datos sintéticos), lo que subraya la importancia de validar contra datos de producción.

7.1 TypeError con dtype nullable de Pandas (Int64) en operaciones de cuantiles de Dask

Problema: al calcular cuantiles (usados en la detección de outliers IQR) sobre la columna UNIDADES (dtype "Int64", el entero nullable de Pandas), Dask lanzaba `TypeError: Cannot interpret 'Int64Dtype()' as a data type`. La causa: `dask.array.percentile` llama internamente a `np.issubdtype(a.dtype, ...)`, y NumPy no reconoce `Int64Dtype()` por no ser un dtype nativo suyo.

Solución: castear la columna a float64 (dtype nativo de NumPy) inmediatamente antes de calcular cuantiles, media o desviación estándar — preservando la semántica de nulos (`pd.NA` se convierte en `NaN`) sin alterar el dtype original de la columna en el DataFrame.

7.2 AttributeError por incompatibilidad del backend dask_expr

Problema: `df["EDAD"].notna()` lanzaba `AttributeError: 'Series' object has no attribute 'notna'` en la versión de Dask instalada (que usa el nuevo backend query-optimizado `dask_expr`).

Solución: reemplazar `.notna()` por `~.isna()`, equivalente funcionalmente y estable en todas las versiones de `dask.dataframe`.

7.3 FutureWarning de pandas en groupby sobre columnas categóricas

Problema: `df.groupby("CANAL")` sobre una columna de dtype "category" generaba un `FutureWarning` sobre el cambio de comportamiento por defecto de `observed=` en versiones futuras de pandas.

Solución: especificar `observed=True` explícitamente.

7.4 Recomputación completa del grafo de tareas de Dask en cada .compute() intermedio

Problema: una primera versión que ejecutaba carga, limpieza, ingeniería de variables y materialización enteramente con Dask tardaba ~939 segundos (15,6 minutos), muy por sobre los ~180 segundos de una versión equivalente en Pandas puro. La causa: cada `.compute()` individual usado para loggear un valor intermedio (conteo de nulos, media, mediana, etc. — más de 10 en total) volvía a ejecutar el grafo de tareas completo desde la lectura del CSV, al no existir ningún mecanismo de cacheo entre llamadas.

Solución: agregar `.persist()` inmediatamente después de la carga, materializando la lectura del CSV una única vez en memoria (como colección Dask, sin salir de su API) antes de encadenar las transformaciones de limpieza e ingeniería de variables. Esto redujo el tiempo total de esas dos etapas a ~90 segundos combinados, dejando el pipeline completo en ~227-234 segundos — comparable a la versión Pandas pura, mientras sí ejercita Dask de forma sustantiva en las tres etapas que más se benefician de procesamiento distribuido (carga, limpieza y feature engineering sobre 3,24 millones de filas).

7.5 OverflowError potencial al calcular EDAD por resta directa de fechas

Problema: un primer intento de calcular EDAD restando directamente dos columnas `datetime64` (FECHA - FECHA_NACIMIENTO) es vulnerable a `OverflowError`: el `Timedelta` resultante se representa internamente en nanosegundos como un entero de 64 bits, cubriendo solo un rango de aproximadamente ± 292 años —

insuficiente ante registros de FECHA_NACIMIENTO con errores de captura muy antiguos, como los que efectivamente existen en el archivo real.

Solución: calcular la edad por componentes (diferencia de años, ajustada según si el mes/día de cumpleaños ya ocurrió respecto a FECHA), una aritmética sobre enteros pequeños inmune a ese desbordamiento.

7.6 UNIDADES constante en todo el dataset real

Hallazgo (no un bug de código): la columna UNIDADES resultó ser exactamente 1 en el 100% de las 3.242.878 transacciones del archivo real, algo que no era evidente a partir de solo leer el enunciado o la muestra de datos del PDF. Esto requirió manejo defensivo consistente en todo el pipeline: exclusión automática y documentada de UNIDADES como predictor en la regresión (produciría VIF infinito), omisión deliberada de su estandarización (evitando una división por cero), y una advertencia explícita en el log de la prueba de hipótesis H2, en vez de reportar silenciosamente un resultado (pendiente=0, p=1) que podría malinterpretarse como evidencia real.

7.7 Experimento con ProcessPoolExecutor: BrokenProcessPool

Ver sección 6.4 para el detalle completo. En síntesis: se evaluó reemplazar ThreadPoolExecutor por ProcessPoolExecutor, con el objetivo de evitar el GIL de Python. El experimento resultó en un crash del pool de procesos por agotamiento de memoria virtual del sistema (Windows, modelo spawn), confirmando empíricamente que ThreadPoolExecutor es la decisión correcta para esta carga de trabajo específica.

7.8 Interrupción operativa de 5 meses en la serie temporal

Ver sección 4.5 para el detalle completo. En síntesis: se detectó un lote aislado de 8 transacciones en un único local en noviembre de 2023, seguido de un extenso período sin actividad en toda la cadena antes del inicio de operaciones sostenidas en marzo/abril de 2024. Consultado el profesor sobre cómo proceder, la indicación fue que cada grupo tome una postura y la justifique explícitamente. La postura adoptada por este equipo: el dataset y todos los análisis que no dependen de continuidad temporal (limpieza, EDA general, hipótesis, regresión, clustering) se mantienen sin modificar; solo la descomposición temporal y el ACF/PACF excluyen ese período inicial (136 días, detectados con un criterio genérico de ventana móvil, no una fecha fija), por ser los dos únicos análisis cuya validez estadística depende de continuidad temporal.

7.9 PORCENTAJE_DESCUENTO fuera del rango declarado

Hallazgo: el enunciado describe PORCENTAJE_DESCUENTO como un valor entre 0 y 1, pero el máximo real observado en los datos es 1,17 (117%). No se modificó ni excluyó este valor (sección 4.1); se documenta como una inconsistencia de calidad de datos a validar con el área de negocio, sin evidencia suficiente en el propio dataset para determinar si es un error de captura o una política de descuentos acumulados/promocionales legítima.

8. Conclusiones

El pipeline implementado procesa exitosamente las 3.242.878 transacciones del dataset de Cruz Morada en aproximadamente 3,9 minutos, combinando Dask (para la carga, limpieza e ingeniería de variables sobre el volumen completo de datos, con `.persist()` como optimización clave) y Pandas/SciPy/statsmodels/scikit-learn (para el análisis estadístico y el modelado, una vez reducido el dataset a un tamaño manejable en memoria). Se implementó además un algoritmo paralelo propio (ThreadPoolExecutor) sobre particiones lógicas por LOCAL, documentando honestamente sus limitaciones de speedup en este entorno específico (Windows, cargas CPU-bound, tareas de grano fino) y validando esa decisión de diseño con un experimento comparativo real (ProcessPoolExecutor).

Entre los hallazgos de negocio más relevantes destacan: (1) UNIDADES es constante (=1) en la totalidad de las transacciones, lo que limita significativamente el valor de esa variable para modelado; (2) existe un segmento pequeño pero muy diferenciado de clientes de alto gasto (3,3% de la base, cluster identificado por K-means) candidato natural para estrategias de fidelización; (3) el canal presencial (POS) y WEB generan, en promedio, tickets mayores que APP (canal de referencia) y CCT, un patrón de canal menos uniforme de lo esperado que amerita revisión con el área comercial; (4) el modelo de regresión, si bien estadísticamente válido, tiene poder predictivo limitado (R^2 ajustado = 0,164), por lo que se recomienda su uso exploratorio y no como predictor definitivo; y (5) se identificaron dos observaciones de calidad de datos (la interrupción operativa de ~4,5 meses en la serie temporal y el máximo de PORCENTAJE_DESCUENTO por sobre el rango declarado) que ameritan validación con el área de negocio antes de tomar decisiones basadas en estos resultados.

El proceso de desarrollo evidenció que muchas de las dificultades más significativas del proyecto no provienen de la teoría estadística o de paralelismo per se, sino de la fricción real entre las abstracciones de estas librerías (dtypes nullable de Pandas, aproximaciones internas de Dask, comportamiento de spawn en Windows) y datos de producción reales, con sus propias inconsistencias — subrayando la importancia de validar todo pipeline contra el dataset real antes de dar por buena cualquier decisión de diseño.